NERMA AI (@NermaAI_bot) — 7-Minute Project Defense Script

Telegram research bot built on GPT Researcher. 5 speakers, one laptop with Telegram on the projector. Button labels are written exactly as they appear in the bot (Russian UI) with the English meaning in brackets. Total speaking time: 7:00.

0. Order and timing

#	Speaker	Topic	Shows on screen	Time
1	Aruzhan	Problem, what NERMA AI is, architecture, request states	`/start`, main keyboard, ℹ️ О боте / Помощь	1:30
2	Erasyl	Settings: 5 modes, language, file formats, per-user isolation	⚙️ Настройки and its menus	1:00
3	Rustam	Live Quick request: clean search query, verified [n] citations, 3 source counters, export on demand	Typed question, status message, result, 📥 Download Files (DOCX / MD)	1:30
4	Madina	Deep Research (pre-run): engine log in chat, topic anchoring, auto-exported DOCX / MD / PDF, Outline	Deep result, files, 📝 Retry as Outline	1:30
5	Nurzhas	Reliability: live cancel, honest `failed`, timeouts, tests, deployment, metrics, closing line	🛑 Cancel Request, failed result, tables	1:30

Before the defense (mandatory): bot is running (`python run_telegram_bot.py`, `TELEGRAM_BOT_TOKEN` and `GOOGLE_API_KEY` set); language switched to 🇬🇧 English in settings so result buttons are in English; already run and visible in chat history: one Deep Research on the Python history topic (takes up to 4 min), one 📝 Retry as Outline on it, and one negative query with a non-existent domain. Only the Quick request (Rustam) and the cancel (Nurzhas) run live.

1. Aruzhan — Problem, product, architecture (1:30)

Shows	Says
Telegram chat with the bot is open, nothing pressed yet.	"NERMA AI is a Telegram bot for sourced research. It is built on the open-source GPT Researcher engine. The user sends a question; the bot searches the web, reads the pages, writes an answer with citations [1], [2], attaches a verified list of URLs and exports it as DOCX, Markdown or PDF."
Still the chat. No button.	"We started from GPT Researcher wrapped in a simple bot and tested it by hand. It failed in four ways: a refusal with zero sources was delivered as a 'Completed report' with files; a request for an official source such as `microsoft.com` was ignored; Deep Research on the history of Python drifted into Python 3.15 release schedules; and one request hung for 40 minutes with no way to cancel. Our project fixes this pipeline and measures the fix."
Sends `/start`. Welcome message and the persistent bottom keyboard with 6 buttons: 🔍 Новое исследование (New research), ⚙️ Настройки (Settings), 📊 Режим отчёта (Report mode), 🌐 Язык ответа (Language), 🎭 Тон изложения (Tone), ℹ️ О боте / Помощь (Help).	"Stack: Python 3.12, `python-telegram-bot`, GPT Researcher, DuckDuckGo for search, Google Gemini as the LLM. Our own code is seven modules in `telegram_bot/`: handlers and pipeline, config, keyboards, progress handler, search-intent extractor, result bundle with exporter, diagnostics. We also changed the engine itself: the DuckDuckGo retriever, the Deep Research prompts and the search timeouts."
Presses ℹ️ О боте / Помощь. Help lists `/start`, `/settings`, `/quick`, `/deep`, `/cancel`, `/help` and defines the 3 source counters.	"Every request goes through explicit states: **queued** → **searching** → **writing** → **exporting** → **completed / partial / failed / cancelled**, shown live in the status message. Each request gets its own id and runs on a frozen copy of the user's settings. Erasyl shows those settings."

Reference for Aruzhan, not read aloud (answer if asked "what is in the code"):

Module	Responsibility
`telegram_bot/bot.py`	Commands, buttons, the research pipeline per mode, status messages, result delivery
`telegram_bot/config.py`	Modes, tones, languages, formats, timeouts from `.env`, per-user settings store with snapshots
`telegram_bot/keyboards.py`	Bottom keyboard and all inline keyboards (settings, cancel, post-report, failed)
`telegram_bot/progress_handler.py`	Streams GPT Researcher log events into the Telegram status message, max one edit per 2 s
`telegram_bot/query_utils.py`	`extract_search_intent()`: short search core, `site:` domains, official-domain requests, search region
`telegram_bot/exporter.py`	`ResearchResultBundle`: [n] ↔ URL reconciliation, refusal detection, DOCX / MD / PDF export
`telegram_bot/diagnostics.py`	Per-request stage timers, failure reasons, structured logs with secret redaction
`gpt_researcher/` (engine changes)	DuckDuckGo retriever: `query_domains`, region, retry; Deep Research: original question in every prompt + drift filter; search timeout; scraper timeout

2. Erasyl — Settings (1:00)

Shows	Says
Presses ⚙️ Настройки. Message "Текущие настройки исследования" with 5 inline buttons: Режим (mode), Тон (tone), Язык (language), Форматы файлов (formats), Сбросить по умолчанию (reset).	"Settings are stored per Telegram user: report mode, tone, output language, file formats. Defaults are Quick mode, Objective tone, Russian, DOCX + MD."
Presses 📊 Режим: … . 5 modes, current one marked ✅ : ⚡ Быстрый обзор (Quick), 📊 Стандартный отчёт, 📚 Подробный отчёт, 🧠 Глубокое исследование (Deep Research), 📝 План исследования (Outline). Presses 🔙 Назад в настройки.	"Five modes. **Quick**: one search, five results, three pages read, answer up to 220 words. **Standard** and **Detailed**: full report with bibliography. **Deep Research**: sub-queries, depth 1, breadth 2. **Outline**: a structured plan with sources instead of an essay."
Presses 🌐 Язык: … , shows 🇷🇺 Русский, 🇬🇧 English, 🇰🇿 Қазақша, confirms 🇬🇧 English is selected. Back, presses 🗂 Форматы файлов: … , toggles ✅ / ⬜ for 📕 PDF, 📘 Word (DOCX), 📄 Markdown (MD); turns PDF on.	"Three output languages; the language is set on the request, not globally, so users do not affect each other. Formats are toggles: in Quick mode files are built only on button press, in all other modes automatically. Changing settings while a request runs does not change that request, because it works on a deep-copy snapshot taken at start. Rustam now runs a live request with these settings."

3. Rustam — Live Quick request (1:30)

Shows	Says
Presses 🔍 Новое исследование, types: `Who founded Microsoft, and in what year? Cite an official microsoft.com source. Answer in two sentences.`	"A hard prompt: instructions plus a required official domain. The intent extractor cuts it to the search core `Who founded Microsoft and in what year`, detects `microsoft.com` and adds a targeted `site:microsoft.com` search. The raw prompt never reaches the search engine."
Status message: ⏳ **Запрос принят в очередь [#id]** (queued), topic, mode, button 🛑 Cancel Request. Lines update: Searching verified sources → Writing concise answer with citations.	"The status message is edited in place with the current state and carries a cancel button bound to this request id."
Result header: ✅ Completed Report, time in seconds, and 3 lines: 🔗 Unique URLs found, 📄 Pages read successfully, 📌 Sources cited in text, plus token cost.	"Three separate counters: URLs returned by search, pages whose full text was actually read, and sources cited in the answer. Not one ambiguous number."
Scrolls to the answer: two sentences with [1], [2] and the list `[1] Title – URL` under it. Points at a `microsoft.com` URL.	"Every [n] is checked against the real source list; unmatched markers and invented URLs are removed. If the official domain were missing, the header would say ⚠️ **Partial Report**, not Completed."
Presses 📥 Download Files (DOCX / MD). DOCX and MD arrive with the request id in the caption. Points at 🔄 Retry in Deep Mode and 📝 Retry as Outline.	"Export is built from the saved result, the research is not re-run; the DOCX has the same text, the same [n] and the same URL list. The same topic can be re-run in another mode with one tap."

4. Madina — Deep Research and automatic export (1:30)

Shows	Says
Scrolls up to the pre-run Deep Research result for: `/deep History of the first Python release (0.9.0 and 1.0), the ABC project at CWI and Guido van Rossum's key design decisions`	"Deep Research builds sub-queries from the question, searches each one, reads the pages and merges one report. Depth 1, breadth 2, total limit 240 seconds. It takes up to 4 minutes, so this one was run before the defense."
Shows the status message of that request: engine steps (Initializing search agent, Querying search providers, sub-queries, Synthesizing answer from sources).	"The progress handler implements GPT Researcher's websocket interface, so the engine's own log streams into the Telegram message. The original question is injected into every planning prompt and a drift filter drops off-topic sub-queries. That is what fixed the drift into Python 3.15."
Header with the 3 counters, then the attached files: PDF, Word (DOCX), Markdown (MD). Opens the DOCX, scrolls to the bibliography.	"Here the counters differ: many URLs found, fewer read, fewer cited. Earlier one number said 45 sources while the bibliography had 14. In non-Quick modes files are exported automatically in the chosen formats. Chat and files come from one `ResearchResultBundle`: same text, same citations, same bibliography."
Shows the pre-run Outline result (from Retry as Outline): a hierarchical plan with sources.	"Outline gives a structured plan of the same topic in one tap, a starting point for a coursework or thesis chapter."

5. Nurzhas — Reliability, tests, deployment, closing (1:30)

Shows	Says
Types `/deep Compare Transformer and LSTM architectures for NLP`, waits for the status message, presses 🛑 Cancel Request. Message changes to 🚫 Request cancelled.	"Cancel sets an event on the active request, cancels its asyncio task and frees the concurrency slot. `/cancel` does the same."
Scrolls to the pre-run negative query: `Find an article about a quantum compiler on site:nonexistent-domain-xyz-99999.org` Result: ❌ failed, no files, buttons 🔄 Retry Query, 📊 Try Standard Mode, ⚙️ Change Settings.	"Zero URLs or a refusal text means `failed`: no 'Completed Report', no files. The baseline exported that refusal as a finished DOCX."
Shows the timeout table below (or the `.env` keys).	"Hard timeouts on the queue, the search, page reading, the LLM call and the whole request: 60 seconds for Quick, 180 standard, 240 deep. No request can hang. Up to 3 requests run in parallel."
Shows the terminal: `pytest tests/test_telegram_bot.py`, 13 passed. Shows `Dockerfile.telegram`, `railway.toml`, `render.yaml`.	"13 automated scenarios with mocked search and LLM, from intent extraction to cancel and timeouts. Deployed as a 24/7 Docker worker on Railway or Render, polling, no public endpoint."
Shows the evaluation table below.	"We measure the result on a held-out set of 5 new topics with 6 metrics: official domain recall, citation precision, false-positive report rate, sub-query on-topic rate, bounded latency, search cost. Result: every citation is verifiable, failures are reported honestly, no request hangs, and the same verified report is in chat, DOCX, Markdown and PDF. Thank you."

Timeouts (seconds, configurable in .env)

Queue wait	Search	Page scrape	LLM call	Total: Quick	Total: Standard / Detailed / Outline	Total: Deep
30	20	12	60	60	180	240

Evaluation: baseline (GPT Researcher + simple bot) vs NERMA AI

Metric	Baseline problem	NERMA AI	Measured as
Official Domain Recall	`query_domains` ignored by the retriever; long prompt broke `site:`	Targeted `site:domain` search, hostname check, `partial` if missing	Share of domain-constrained queries whose bibliography has that domain
Citation Precision	[n] in DOCX without URL list; 45 "sources" vs 14 in bibliography	Every [n] mapped to `[n] Title — URL`, invented URLs removed	Confirmed [n] / all [n]
False-Positive Report Rate	Refusal with 0 sources labelled Completed and exported	0 URLs or refusal → `failed`, 0 files	Empty-search queries marked completed (target 0%)
Sub-query On-Topic Rate	Historical question drifted to Python 3.15–3.16	Original question in every prompt + drift filter	Sub-queries inside the question's scope
Bounded Latency & Cancellation	Requests hung 10–45 min, no cancel	Per-stage and total timeouts, cancel button	Median / max response time, cancel reaction time
Search & Cost Efficiency	Quick searched twice and exported files every time	One search; export only on button, from cache	Searches and $ per Quick request

Dev set: 7 scenarios used while fixing the bot. Held-out set: 5 new topics (PyTorch, Django, Transformer vs LSTM, Linux 0.01, RAG metrics) for the final measurement, so the heuristics are not tuned to the demo questions.

Likely questions — who answers

Question	Answers	Answer
Why DuckDuckGo and Gemini?	Aruzhan	No API key needed for search; Gemini free tier for the LLM. Both are set in `.env` and can be swapped for any GPT Researcher retriever or LLM provider.
What did you change in GPT Researcher itself?	Aruzhan	DuckDuckGo retriever now supports `query_domains` and region; Deep Research prompts carry the original question and no longer force "recent developments"; search and scraper got timeouts.
How do you know a citation is real?	Rustam	The LLM only sees numbered sources from our list. After generation each [n] is matched to that list; unmatched markers and URLs not in the list are deleted before anything is shown or exported.
Difference between partial and failed?	Madina	`failed`: no sources or a refusal, nothing exported. `partial`: a sourced answer exists but the requested official domain was not found; the user is warned explicitly.
Where are settings stored? Several users at once?	Erasyl	In-memory store keyed by Telegram user id; each request runs on a deep-copy snapshot, so concurrent users and mid-run changes do not interfere. Up to 3 requests run in parallel.
Is this an improvement of a model or of a dataset?	Nurzhas	An improvement of the RAG pipeline measured on a Dev / Held-Out query dataset we built. If weights must be trained: collect prompt → search-query and claim → source pairs from the diagnostics logs and fine-tune a small reranker or a LoRA adapter.